

Trabajo Práctico

Implementación de una aplicación demostrativa basada en Inteligencia Artificial



UNIVERSITAT
ROVIRA i VIRGILI
Fundació URV

Artificial Intelligence Foundations

Microcredencial — Fundació Universitat Rovira i Virgili

Profesor: Enrique Molina Giménez

Abril 2026

CATEGORIA:

IA GENERATIVA

DESCRIPCIÓN DEL RETO:

GENERADOR DE RECETAS A PARTIR DE INGREDIENTES DISPONIBLES

ALUMNOS:

IMMA ABAD NEBOT y MANEL GONZÁLEZ PÉREZ

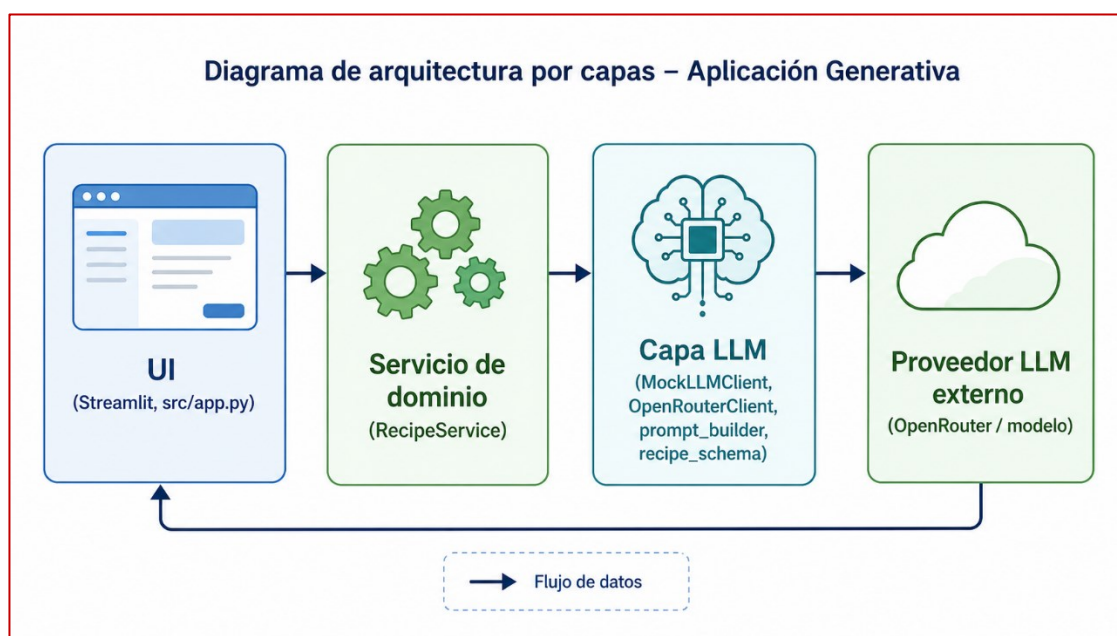
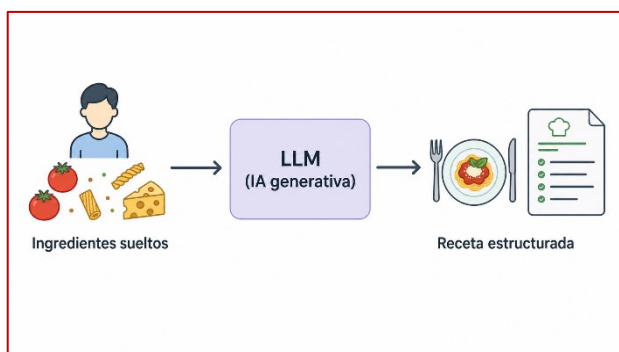
ÍNDICE

ABSTRACT.....	2
INTRODUCCIÓN: PUESTA EN ESCENA DEL PROBLEMA A RESOLVER.....	3
IMPLEMENTACIÓN DE LA APLICACIÓN: DESCRIPCIÓN DETALLADA DE LAS VISTAS Y FUNCIONALIDADES OFRECIDAS	5
INTEGRACIÓN CON LLM: MODELO UTILIZADO (Y JUSTIFICACIÓN DE LA ELECCIÓN), FLUJO DE COMUNICACIÓN APP/LLM, ESTRUCTURA DE LAS ENTRADAS (PROMPTS) Y SALIDAS (OUTPUTS).....	10
CONCLUSIÓN: BREVE REFLEXIÓN FINAL SOBRE EL RETO	15

ABSTRACT

Este trabajo presenta una aplicación web desarrollada con Streamlit que genera recetas de cocina estructuradas a partir de los ingredientes disponibles en casa, utilizando una IA Generativa (LLM) accesible vía API. El usuario introduce una lista libre de ingredientes y la aplicación obtiene una propuesta de receta con título, raciones, lista de ingredientes, tiempo aproximado, dificultad, pasos detallados y un campo de “cadena de pensamiento” interno para mejorar la calidad culinaria.

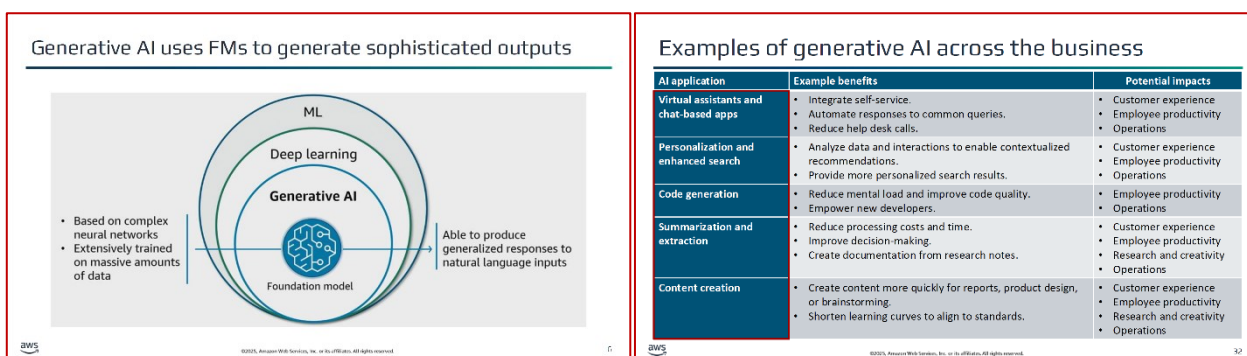
El proyecto se centra en dos aspectos clave: por un lado, la integración técnica con un LLM externo mediante una capa de servicio y cliente especializada; por otro, el diseño de *prompts* estructurados que fuerzan una salida en formato JSON validable, lo que permite mapear los resultados a componentes visuales de Streamlit en lugar de mostrar texto plano. La solución incluye también un modo *mock* para desarrollo local y una batería de pruebas unitarias básicas.



INTRODUCCIÓN: PUESTA EN ESCENA DEL PROBLEMA A RESOLVER

El problema que se aborda en este reto es frecuente en la vida cotidiana: muchas personas disponen de ingredientes disponibles en casa pero no saben qué cocinar con ellos o cómo combinarlos de forma equilibrada y rápida. Al mismo tiempo, los modelos de lenguaje generativos son capaces de proponer recetas originales y detalladas a partir de una simple descripción textual de los ingredientes disponibles.

En términos del módulo *Introducing Generative AI*, nuestra aplicación se apoya en un modelo fundacional de tipo LLM especializado en tareas de lenguaje, capaz de generar texto a partir de *prompts* en lenguaje natural. Esto nos permite reutilizar un modelo preentrenado en lugar de construir uno desde cero.



Desde el inicio del proyecto nos planteamos el reto combinando esta necesidad cotidiana con una división clara de responsabilidades técnicas en el desarrollo. A nivel de diseño, acordamos separar el trabajo en dos bloques principales:

1. En el **backend**, empezar por una prueba de concepto con Streamlit para comprobar la integración con alguna API gratuita del **repositorio de free LLM APIs**, utilizando inicialmente un “*frontend* de cartón-piedra”. En paralelo, definir un contrato claro entre *frontend* y *backend* y construir una respuesta *mock* estable, de manera que, aunque siempre devolviera la misma receta, nos permitiera validar que la respuesta se podía parsear y mostrar visualmente.
2. A continuación, una vez verificada la integración, **formalizar el contrato entre backend y LLM** (entrada: lista de ingredientes; salida: receta estructurada), aplicando *prompt engineering* para que el modelo respetara un JSON Schema concreto. Esto implicaba añadir validación del JSON devuelto, así como una política básica de gestión de errores y reintentos.
3. En el **frontend**, el foco se sitúa en el parseo de la respuesta JSON a elementos visuales: presentar la receta en distintos componentes gráficos e interactivos (título, raciones, listas de ingredientes y pasos), evitando mostrar la salida del LLM como texto en bruto y cumpliendo el requisito del reto de mapear el *output* a la interfaz.

Nuestro objetivo es, por tanto, doble: por un lado, demostrar cómo se puede encapsular la capacidad generativa de un LLM dentro de una aplicación sencilla, accesible desde el navegador, que actúa como un “chef virtual” capaz de generar recetas coherentes, estructuradas y fácilmente legibles a partir de la lista de ingredientes introducida por la persona usuaria; por otro, poner en práctica los conceptos trabajados en la microcredencial (modelos fundacionales, prompt engineering, integración con LLMs y responsabilidad en IA generativa) aplicados a un caso de uso concreto del reto 7: generador de recetas a partir de ingredientes disponibles.

Al tratarse de IA generativa, también hemos tenido en cuenta algunos de los riesgos descritos en el módulo sobre uso responsable (alucinaciones, sesgos, toxicidad), aplicando validación de salidas y acotando el dominio a recetas de cocina.

Responsible AI

- An approach to building and using AI systems that encompasses the following principles:

- People-centric
- Ethical
- Safe
- Unbiased



IMPLEMENTACIÓN DE LA APLICACIÓN: DESCRIPCIÓN

DETALLADA DE LAS VISTAS Y FUNCIONALIDADES OFRECIDAS

Desde el inicio del proyecto acordamos abordar el reto combinando la necesidad funcional con una división clara de responsabilidades técnicas. A nivel de diseño, nos planteamos el trabajo en dos bloques principales. En el **backend**, empezar por una prueba de concepto con Streamlit para comprobar la integración con alguna API gratuita de LLM, definiendo desde el principio un contrato JSON entre *frontend* y *backend* y construyendo una respuesta *mock* estable que siempre devolviera la misma receta; el objetivo de este *mock* era poder validar que la estructura se podía parsear y representar visualmente, incluso antes de tener una integración real con un modelo generativo. En el **frontend**, el foco se situó en convertir esa respuesta JSON en componentes gráficos de Streamlit (título, raciones, listas de ingredientes y pasos), evitando mostrar la salida del LLM en bruto y cumpliendo el requisito del reto de mapear el *output* a la interfaz.

Como **primer prototipo**, la aplicación se implementó con un único fichero `app.py` en la raíz del repositorio, que concentraba toda la lógica en un solo *script*. Ese archivo definía la interfaz básica de Streamlit (área de texto para ingredientes, botón para generar receta y checkbox para activar la respuesta simulada), construía directamente el prompt para el LLM, gestionaba tanto el modo *mock* como la llamada real a la API y se encargaba de parsear la respuesta JSON y mostrarla en pantalla. Aunque esta solución permitía disponer rápidamente de una demo funcional, mezclaba responsabilidades de interfaz, integración con el LLM y validación de datos, lo que dificultaba las pruebas y la evolución modular de la aplicación.

Generador de recetas a partir d'ingredients

Introdueix els ingredients que tens disponibles i genera una recepta. Pots fer servir una resposta simulada (mock) o, quan estigui configurada, una crida real a un model de llenguatge (LLM) via OpenRouter.

Escriu aquí els ingredients que tens disponibles:

pa, oli, carn

☒ Fer servir resposta simulada (mock) ⓘ

Generar recepta

JSON de la recepta

```
{
  "title": "Pasta amb salsa de tomàquet ràpida",
  "servings": 2,
  "ingredients": [
    "200 g de pasta seca",
    "2 tomàquets madurs",
    "50 g de formatge ratllat",
    "2 cullerades d'oli d'oliva",
    "Sal i pebre al gust"
  ],
  "time_minutes": 20,
  "difficulty": "Easy",
  "steps": [
    "Bull la pasta en una olla amb aigua i sal fins que sigui al punt.",
    "Mentrestant, talla els tomàquets a dauets i salta'ls en una paella amb oli d'",
    "Salpebra la salsa de tomàquet i barreja-la amb la pasta escorreguda.",
    "Serveix al plat i afegeix el formatge ratllat per sobre."
  ]
}
```

Pasta amb salsa de tomàquet ràpida

Racions: 2

Temps aproximat: 20 minuts

Nivell de dificultat: Easy

Ingredients

- 200 g de pasta seca
- 2 tomàquets madurs
- 50 g de formatge ratllat
- 2 cullerades d'oli d'oliva
- Sal i pebre al gust

Passos

1. Bull la pasta en una olla amb aigua i sal fins que sigui al punt.
2. Mentrestant, talla els tomàquets a dauets i salta'ls en una paella amb oli d'oliva.
3. Salpebra la salsa de tomàquet i barreja-la amb la pasta escorreguda.
4. Serveix al plat i afegeix el formatge ratllat per sobre.

[ia-recetas/app.py at main · iabad5-del/ia-recetas](#)

[Generador de recetas · Streamlit](#)

En la rama `segona_iteracio` se ha refactorizado el proyecto para pasar de ese *script* monolítico a una **arquitectura modular por capas** dentro de la carpeta `src/`. El antiguo `app.py` de raíz desaparece y se sustituye por un nuevo `src/app.py` centrado exclusivamente en la **interfaz de Streamlit**, mientras que la lógica de negocio se traslada a un **servicio de dominio** `RecipeService`; la **integración con el LLM** se encapsula en **clientes separados** (`MockLLMClient` para el modo simulación y `OpenRouterClient` para llamadas reales), y la **validación de la respuesta** se basa en un esquema JSON independiente (`recipe_output.schema.json`) con utilidades específicas para recorrer y renderizar los campos. Esta reorganización introduce también una **capa de dominio** (`Recipe`), un **módulo de construcción de prompts** (`prompt_builder`) y una **batería de tests** unitarios que cubren configuración, cliente *mock*, esquema y servicio, mejorando la mantenibilidad, las pruebas y la capacidad de evolucionar la solución sin afectar a la interfaz de usuario.

[Github.com/ArcadiaLliure/ia-recetas/tree/segona_iteracio](https://github.com/ArcadiaLliure/ia-recetas/tree/segona_iteracio)

[Generador de recetas · Streamlit](#)

The screenshot shows a code editor interface. On the left, a file explorer displays the project structure: `src/` containing `__pycache__`, `config.cpython-313.pyc`, `exceptions.cpython-313.pyc`, `app.py`, `config.py`, `exceptions.py`, `llm`, `__pycache__`, `client.cpython-313.pyc`, `client.py`, `tests`, `confest.py`, and `test_config.py`. The main editor area shows the content of `src/app.py`, which includes imports for `from __future__ import annotations` and `import json`. The code is highlighted with syntax coloring, and the editor shows line numbers and a search bar at the top.

La aplicación se ha implementado con Streamlit como *framework* de *frontend*, siguiendo la directriz del enunciado de utilizarlo como marco común para las demos del curso. El punto de entrada `src/app.py` define la interfaz principal:

Cuando la persona usuaria pulsa el botón, la app construye un `RecipeService` mediante inyección de dependencias: si el modo *mock* está activado se utiliza `MockLLMClient`; en caso contrario se instancia `OpenRouterClient` con la configuración leída de variables de entorno (por ejemplo `LLM_API_KEY` y parámetros de modelo). El servicio valida que la entrada no esté vacía y delega la generación en la capa LLM. La respuesta recibida (un JSON) se valida contra el esquema `recipe_output.schema.json` y, si es correcta, se convierte en un objeto de dominio `Recipe` antes de devolverse a la interfaz.

Generador de recetas a partir de ingredientes

Introduce los ingredientes que tienes disponibles para generar una receta. Puedes usar modo mock o una llamada real a OpenRouter.

Ingredientes disponibles

tomate, pan, carne

☐ Usar respuesta simulada (mock) ⓘ

Temperatura del LLM

0.6

Generar receta

> Ver JSON de la receta

Carne en Salsa de Tomate con Pan

Raciones: 2

Ingredientes

- 300g de carne (res o cordero) en trozos
- 2 tomates grandes maduros
- 4 rebanadas de pan
- 2 cucharadas de aceite de oliva
- 1 diente de ajo (opcional)
- Sal al gusto
- Pimienta al gusto

Tiempo aproximado (minutos): 30

Dificultad: Easy

Pasos

1. Corta la carne en trozos pequeños y sazónala con sal y pimienta.
2. Calienta el aceite en una sartén a fuego medio.
3. Añade la carne y cocínala hasta que se dore por todos lados (unos 5-7 minutos).
4. Mientras tanto, corta los tomates en cubos pequeños.
5. Agrega los tomates a la sartén con la carne y mezcla bien.
6. Reduce el fuego a medio-bajo y cocina por 10-12 minutos hasta que los tomates se desmenen y formen una salsa.
7. Prueba y ajusta la sal si es necesario.
8. Tuesta las rebanadas de pan en la misma sartén o en un tostador.
9. Sirve la carne en salsa de tomate caliente acompañada del pan tostado para mojar.

Un área de texto donde la persona usuaria introduce los ingredientes disponibles en lenguaje natural (por ejemplo: “tomate, pasta, queso, aceite de oliva”).

Un checkbox que permite alternar entre un modo *mock* (respuesta simulada con un JSON determinista) y la llamada real a un LLM vía API.

Un control deslizante para configurar la temperatura del modelo, que permite ajustar el grado de creatividad entre valores bajos (respuestas más deterministas y repetibles) y altos (respuestas más variadas). Este parámetro se pasa internamente al servicio de recetas y desde allí al cliente del LLM, siguiendo las recomendaciones del curso sobre ajuste de parámetros de inferencia para equilibrar precisión y diversidad en los resultados generados.

Un botón “Generar receta” que desencadena el flujo de generación.

Un *expander* que permite ver el JSON completo para depuración, lo que facilita la comprobación de que el contrato de datos con el LLM se cumple.

En la interfaz, la receta se muestra de forma estructurada

En la interfaz, la receta se muestra de forma estructurada: renderizado de los campos según los metadatos del esquema (`x-ui-widget`, `x-ui-list-style`, títulos amigables), distinguiendo listas numeradas para los pasos y listas con viñetas para los ingredientes, y utilizando encabezados para el título de la receta y otros campos destacados.

Campos adicionales opcionales, como la cadena de pensamiento (`chain_of_thought`), pueden ocultarse o mostrarse según las reglas definidas en `recipe_schema`, lo que permite aprovechar el razonamiento interno del modelo sin saturar la interfaz si no es necesario.

La aplicación gestiona distintos tipos de error mediante excepciones específicas: errores de validación de entrada, mala configuración de la API (por ejemplo ausencia de API key), errores HTTP en la llamada al LLM y respuestas no parseables. En este último caso se ofrece la opción de visualizar la respuesta cruda del LLM desde la propia interfaz para facilitar el diagnóstico. En conjunto, esta arquitectura por capas (UI, servicio de dominio, cliente LLM y validación de esquema) sigue las buenas prácticas vistas en el módulo de implementación de aplicaciones generativas y deja preparada la base para futuras extensiones de funcionalidad.

Generador de recetas a partir de ingredientes

Introduce los ingredientes que tienes disponibles para generar una receta. Puedes usar modo mock o una llamada real a OpenRouter.

Ingredientes disponibles

tomate, pan, carne

☒ Usar respuesta simulada (mock) ⓘ

Temperatura del LLM

0.1 0.2 1.0

Generar receta

> Ver JSON de la receta

Pasta con salsa de tomate rapida

Raciones: 2

Ingredientes

- 200 g de pasta
- 2 tomates maduros
- 2 cucharadas de aceite de oliva
- 50 g de queso rallado
- Sal y pimienta al gusto

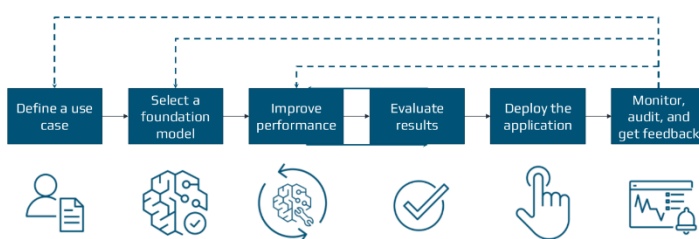
Tiempo aproximado (minutos): 20

Dificultad: Easy

Pasos

1. Hierva la pasta en agua con sal hasta que quede al dente.
2. Sofríe los tomates picados con aceite de oliva durante 8 minutos.
3. Mezcla la pasta escurrida con la salsa de tomate y ajusta sal y pimienta.
4. Sirve caliente con queso rallado por encima.

Generative AI application lifecycle



El paso de un *script* monolítico a una arquitectura por capas refleja el ciclo de vida propuesto en el módulo “*Implementing Generative AI Applications*”: definir el caso de uso, crear un prototipo sencillo y, posteriormente, mejorar la solución con una arquitectura más robusta y testeable antes de desplegarla.

Selecting the foundation model (FM)

- Model capabilities
- Model size and complexity
- Inference speed
- Costs
- Information about its training data
- Ethical considerations
- Implementation considerations

Model depends on our needs



AWS

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

10

Improving performance

- Adjust inference parameters
- Perform prompt engineering
- Implement RAG
- Customize the model through fine-tuning or continued pre-training



AWS

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

11

Evaluate (and iterate)

- Align evaluation criteria with the use case.
- Use a combination of evaluation methods
- Consider both quantitative metrics and qualitative assessments.
- Continuously evaluate throughout the lifecycle.
- Adapt evaluation strategies as the model and its applications evolve.



AWS

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

10

Use a combination of evaluation techniques



Employ human evaluation

- Particularly useful for assessing qualitative aspects such as relevance, coherence, and appropriateness of outputs



Choose from popular benchmarks

- General Language Understanding Evaluation (GLUE)
- SuperGLUE
- Stanford Question Answering Dataset (SQuAD)



Run automated metrics

- Perplexity
- Bilingual Evaluation Understudy (BLEU)

AWS

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

11

Deploy the application

- Configure infrastructure and integrate with other application components.
- Secure the environment and application.
- Setup monitoring.
- Prepare staff and implement change management.



AWS

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

12

Monitoring, auditing, and feedback loops

Key metrics to monitor

- Compute resources, network performance, storage usage
- System logs
- Model performance metrics
- Bias and fairness metrics
- Compliance adherence

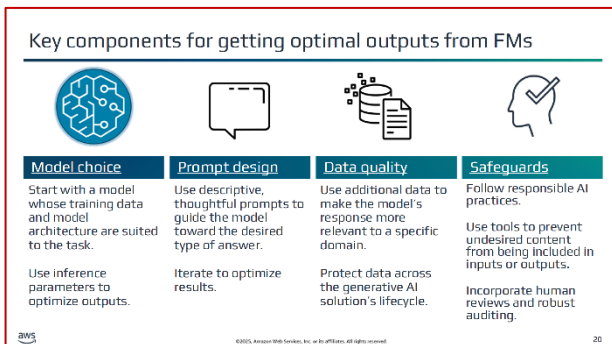


AWS

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

13

INTEGRACIÓN CON LLM: MODELO UTILIZADO (Y JUSTIFICACIÓN DE LA ELECCIÓN), FLUJO DE COMUNICACIÓN APP/LLM, ESTRUCTURA DE LAS ENTRADAS (PROMPTS) Y SALIDAS (OUTPUTS)



La aplicación integra un modelo de LLM externo a través de una arquitectura en capas que separa claramente la lógica de interfaz, el servicio de dominio y el cliente de acceso a la API. Esta elección sigue los criterios de selección de modelos fundacionales vistos en el módulo “*Working with Foundation Models*”: capacidades para la tarea de generación de texto, coste y límites de uso, facilidad de

integración y esfuerzo de despliegue.

Para identificar proveedores adecuados utilizamos el repositorio [Free LLM API Resources](#), que recopila distintos servicios con acceso gratuito o de bajo coste a LLMs vía API.

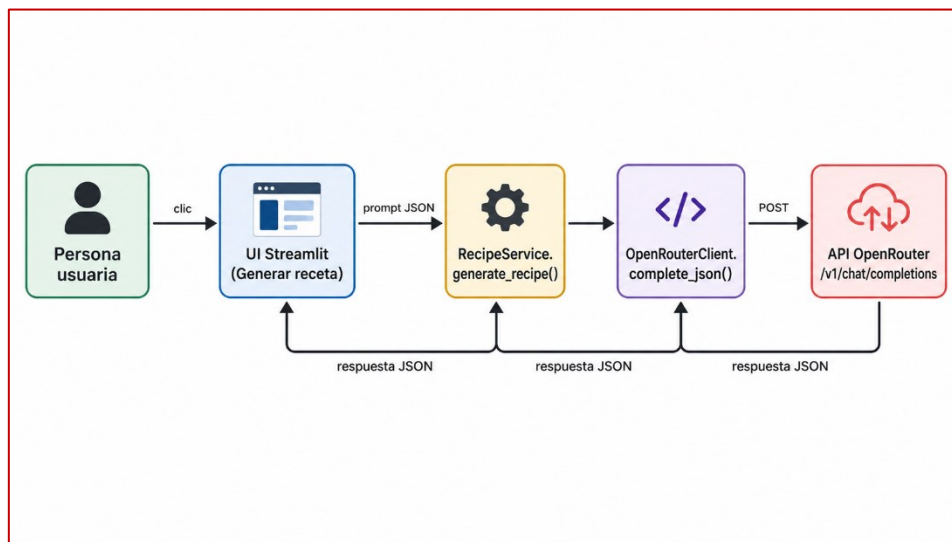
En la capa de presentación (`src/app.py`), la función `build_recipe_service()` construye un objeto `RecipeService` inyectando una implementación de cliente LLM: `MockLLMClient` en modo simulación o `OpenRouterClient` para llamadas reales. Esta decisión se basa en el patrón de inyección de dependencias, que permite desacoplar Streamlit de los detalles de comunicación con el modelo generativo y simplifica tanto las pruebas como el cambio de proveedor.

A nivel de **modelo**, se ha optado por utilizar **OpenRouter** como pasarela hacia LLMs compatibles con la API de OpenAI, configurando un modelo de propósito general orientado a tareas de conversación y generación de texto. Esta elección se justifica por varios motivos: la existencia de un *endpoint* estándar (`/api/v1/chat/completions`), la posibilidad de seleccionar modelos con política de precios flexible o nivel gratuito, y la facilidad de integración con bibliotecas HTTP sencillas como `requests`, en línea con los criterios del curso para elegir modelos fundacionales (capacidades, coste, facilidad de despliegue e integración).

El **flujo de comunicación app/LLM** es el siguiente:

1. La capa de servicio construye el *prompt* llamando a `build_recipe_prompt(ingredients)`, donde se incorporan tanto las instrucciones de alto nivel como el esquema JSON deseado.

2. El cliente `OpenRouterClient` recibe el *prompt* y compone una petición HTTP POST al *endpoint* `/api/v1/chat/completions`, incluyendo el nombre del modelo, la temperatura y la API key leída de la configuración.
3. La respuesta de la API se recibe como texto; se asume, gracias al diseño del *prompt*, que dicho texto es un objeto JSON único con la estructura de receta esperada.
4. Este JSON se parsea y se valida contra `recipe_output.schema.json`. Si es válido, se transforma en un objeto `Recipe` y se devuelve al *frontend* para su renderizado; si no lo es, se lanza una excepción `InvalidLLMResponseError` y se notifica al usuario con la opción de inspeccionar la respuesta cruda.
5. En paralelo, se mantiene un cliente `MockLLMClient` que implementa la misma interfaz (`complete_json(prompt)`) pero devuelve siempre una receta determinista y válida. (Este cliente permite desarrollar y probar la app sin depender de la disponibilidad o cuotas del proveedor LLM, y es especialmente útil para los tests unitarios y la demo en caso de incidencias con la API real.)



Esta aproximación sigue las recomendaciones del módulo de “*Foundation Models*”: reutilizar un modelo preentrenado “off-the-shelf”, ajustar parámetros de inferencia y centrar los esfuerzos en el diseño de *prompts* y en la integración responsable, en lugar de entrenar un modelo propio desde cero.

PROMPT ENGINEERING: JUSTIFICACIÓN DE LAS TÉCNICAS UTILIZADAS Y SU IMPACTO EN LA CALIDAD DE LOS RESULTADOS

El componente [llm/prompt_builder.py](#) encapsula la lógica de *prompt engineering* aplicada al reto. El *prompt* resultante incorpora varias de las técnicas presentadas en el módulo “Using Prompts and Prompt Engineering”.

```

src/llm/prompt_builder.py
5 5
6 6 def build_recipe_prompt(ingredients: str) -> str:
7 - """Build the prompt sent to the LLM from an ingredients list."""
8 + """
9 + JUSTIFICACIÓ DE LES TÈCNIQUES DE PROMPT ENGINEERING APLICADES:
10 + Aquest prompt ha estat dissenyat per maximitzar la qualitat culinària i garantir
11 + l'estabilitat del parseig de dades en el backend, aplicant aquestes tècniques:
12 +
13 + 1. Etiquetatge (XML Tags): L'ús d'etiquetes (<role>, <instructions>, etc.) estructura
14 + clarament les parts del prompt. Això ajuda el model a distingir ràpidament el
15 + context de les instruccions i les regles estrictes.
16 +
17 + 2. Role-playing, To i Audiència (<role>): S'assigna una personalitat experta
18 + ("professional chef AI"), un to ("enthusiastic") i una audiència clara
19 + ("beginner cooks"). Això assegura que la resposta tingui un vocabulari
20 + adequat, pedagògic i amb molt de sentit culinari.
21 +
22 + 3. Descripció Clara i Concisa (<instructions>): S'especifica exactament què ha de
23 + fer la IA ("create ONE cooking recipe"), evitant ambigüitats i forçant-la a
24 + treballar estrictament amb els ingredients de l'usuari.
25 +
26 + 4. Cadena de Pensament / Chain of Thought (CoT): S'instrueix el model a raonar
27 + pas a pas abans de donar la recepta (analitzar, pensar el perfil de sabors i
28 + planificar). Aquest procés s'encapsula com a primer camp dins de l'esquema
29 + JSON ("chain_of_thought") per obligar la IA a reflexionar i millorar la
30 + qualitat de la recepta, sense trencar el format de dades.
31 +
32 + 5. Sortida Estructurada (<json_schema>): S'imposa un contracte de dades rígid.
33 + És indispensable per garantir la interoperabilitat i poder mapejar els valors
34 + en diferents components de Streamlit, complint la norma del Repte 7 de no
35 + mostrar el text en brut.
36 +
37 + 6. Single-shot Prompting (<example>): Es proporciona un exemple complet d'un
38 + JSON vàlid. Mostrar a la IA una demostració d'allò que s'espera redueix
39 + dràsticament la probabilitat que s'equivoqui en el format (com ara llistes
40 + d'ingredients o de passos).
41 +
42 + 7. Negative Prompting i Restriccions (<rules>): S'indica de forma explícita què
43 + TÉ PROHIBIT fer ("Do NOT include any text before or after..."). Actua com a

```

```

43 +     salvaguarda fonamental per evitar al·lucinacions conversacionals prèvies
44 +     (ex. "Aquí tens la teva recepta!") que trencarien la funció `json.loads()`.
45 +     -----
46 +
47 +     """
8  48     recipe_schema = recipe_output_schema_as_text()
9  49     return f"""<role>
10 50     You are a professional chef AI. Your tone is enthusiastic and encouraging.
11 51     Your audience are beginner cooks who need simple, easy-to-follow explanations.
12 52     </role>
13 53
14 54     <instructions>
15 55     Your task is to create ONE cooking recipe using mainly the following available ingredients:
16 56     {ingredients}
17 57
18 58     Before creating the recipe, think step-by-step about flavor profiles, ingredient combinations, and the cooking method.
19 59     </instructions>
20 60
21 61     <json_schema>
22 62     Return the recipe STRICTLY as a single JSON object that validates against this JSON Schema:
23 63     {recipe_schema}
24 64     </json_schema>
25 65
26 66     <rules>
27 67     - Use mostly the ingredients provided, you can add a few basic pantry items if needed (salt, oil, etc.).
28 68     - Make sure the JSON is valid and can be parsed by a JSON parser.
29 69     - Do NOT include any text before or after the JSON.
30 70     - Do NOT include comments in the JSON.
31 71     - Answer ONLY with the JSON object.
...

```

En primer lugar, se estructura el mensaje usando etiquetas similares a XML (<role>, <instructions>, <json_schema>, <rules>), lo que proporciona al modelo una **separación clara entre contexto, tarea, contrato de salida y restricciones**. Esta estructura aplica las buenas prácticas de diseño de *prompts* descritas en el módulo “*Using Prompts and Prompt Engineering*”: ser claros y concisos, incluir contexto relevante, definir el formato de respuesta y personalizar el rol, tono y audiencia.

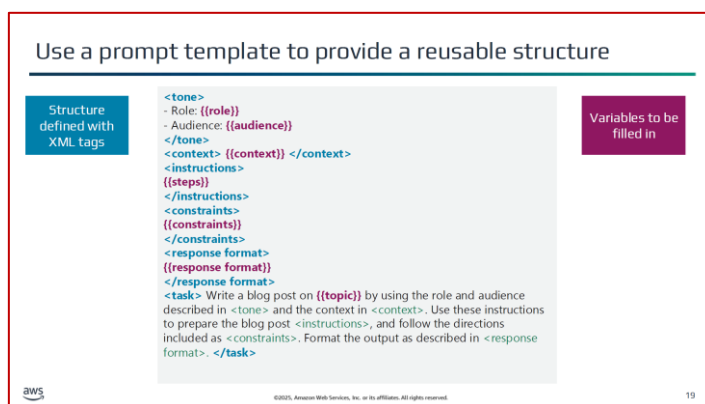
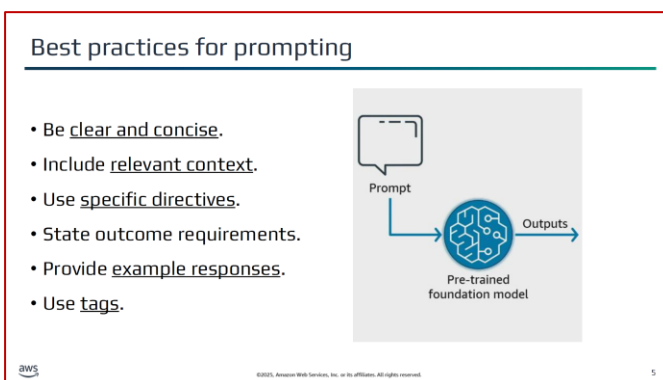
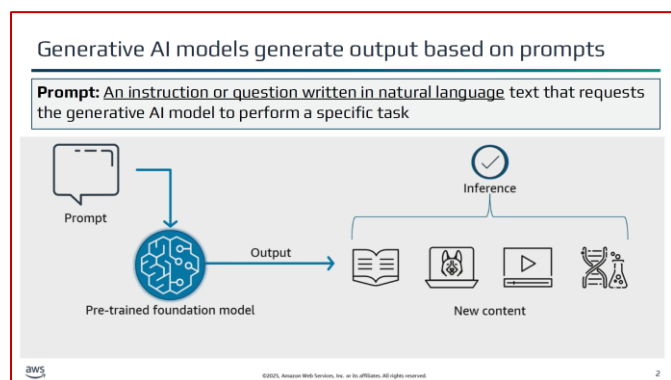
Dentro de <role> se define al modelo como “professional chef AI” con un tono entusiasta y una audiencia de cocineros principiantes, aplicando las recomendaciones de **definir rol, tono y audiencia** para personalizar el estilo de respuesta.

En el bloque <instructions> se **describe de forma explícita y concisa la tarea**: generar una única receta utilizando principalmente los ingredientes proporcionados y razonando paso a paso sobre perfiles de sabor, combinaciones y técnica de cocción. Este razonamiento se refleja en el campo `chain_of_thought` del JSON de salida, lo que aplica una variante controlada de *chain-of-thought prompting*, donde se anima al modelo a razonar paso a paso antes de producir la salida estructurada, **sin exponer el razonamiento al usuario final**.

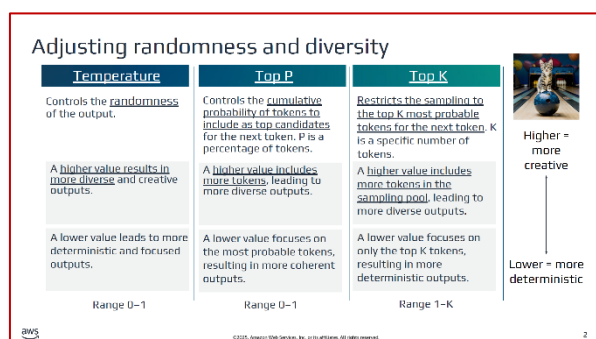
El bloque <json_schema> incluye el texto del JSON Schema completo de salida (`recipe_output.schema.json`), de modo que el modelo ve no solo ejemplos aislados sino la definición formal de los campos, tipos y restricciones. (Incluir el JSON Schema completo corresponde a una técnica

de *structured output prompting*, que mejora la consistencia del formato y reduce la necesidad de *post-processing* complejo). Esto refuerza el **uso de salida estructurada**, recomendado en las buenas prácticas para obtener formatos predecibles y fácilmente parseables.

Por último, en `<rules>` se emplea *negative prompting* para prohibir comportamientos que romperían el parseo: añadir texto antes o después del JSON, incluir comentarios o desviarse del formato. Este tipo de restricciones concuerdan con las **estrategias vistas en el curso para reducir alucinaciones y asegurar que la salida es utilizable por el sistema**, sin necesidad de postprocesados frágiles. (Las reglas que prohíben texto fuera del JSON y comentarios son un ejemplo de *negative prompting*, tal y como se ilustra en los ejemplos de malas prácticas de *prompting* del módulo.)



Además del *prompt*, la aplicación ajusta parámetros de inferencia del LLM, como la **temperatura**, que controla el equilibrio entre respuestas más conservadoras y respuestas más creativas. Este control se expone al usuario a través de un deslizador en la interfaz y se inyecta en la configuración del cliente de OpenRouter, en línea con las recomendaciones del módulo sobre cómo adaptar un modelo fundacional al caso de uso concreto.



CONCLUSIÓN: BREVE REFLEXIÓN FINAL SOBRE EL RETO

El trabajo ha permitido desplegar un generador de recetas a partir de ingredientes disponibles que integra de forma efectiva un modelo de lenguaje generativo en una aplicación web sencilla.

La separación en capas (interfaz en Streamlit, servicio de dominio, cliente LLM, validación mediante esquema) facilita tanto el desarrollo como las pruebas y la futura evolución del sistema.

El uso de un *prompt* cuidadosamente estructurado y orientado a salidas JSON ha sido clave para cumplir con el requisito de parsear y mapear el resultado a componentes de la interfaz, en lugar de mostrar texto libre.

Entre las limitaciones detectadas se encuentran la dependencia de un proveedor externo de LLM, las posibles restricciones de cuota en los modelos gratuitos y el hecho de que, puntualmente, el modelo puede seguir generando salidas no válidas pese a las restricciones del *prompt*. Además, la aplicación actual se centra en un flujo sencillo de ingredientes → receta y no contempla todavía preferencias dietéticas avanzadas, múltiples propuestas o ajustes interactivos. (Estas limitaciones se alinean con los retos identificados en los módulos de “IA responsable y de seguridad para GenAI”: dependencia de proveedores externos, gestión de cuotas, calidad variable de las salidas y necesidad de incorporar salvaguardas adicionales.)

Como líneas de trabajo futuro se proponen: añadir más parámetros de entrada (tiempo máximo disponible, restricciones dietéticas, nivel de habilidad del usuario), incorporar mecanismos adicionales de control responsable de la IA (filtros de contenido, comprobaciones básicas de seguridad alimentaria, etc.), y explorar otros modelos o proveedores con mejor soporte multilingüe y garantías de uso responsable, siguiendo las dimensiones de IA responsable y de gobernanza tratadas en el curso. En conjunto, las mejoras propuestas encajan con las fases de monitorización, evaluación continua y retroalimentación del ciclo de vida de aplicaciones generativas presentado en el módulo “Implementing Generative AI Applications”.

